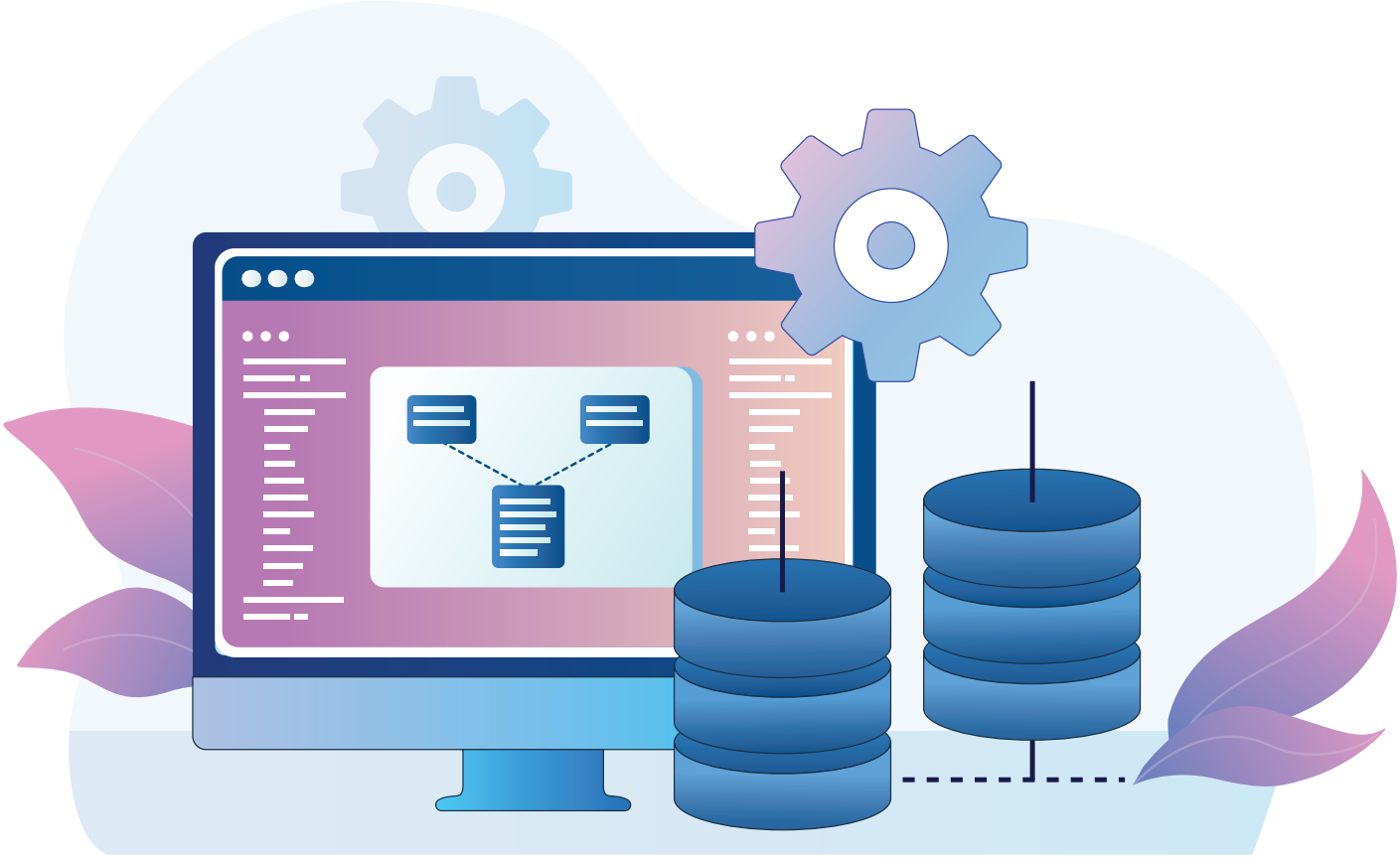
Joins and Foreign Keys

05

For comprehensive and accurate data analysis, ensuring data integrity and efficiently accessing related information across multiple sources is essential.



05. Joins and Foreign Keys

Recall:

Before getting started, quickly recap the DBMS concepts:

- **Database:** A structured or organised collection of related data, typically stored in an electronic system.
- **Table:** A structured collection of related data organised in rows(records) and columns(fields).
- **DBMS or Database Management System:** A software package that enables users to create, maintain, and use a database.
- **RDBMS or Relational Database Management System:** A type of DBMS that provides a more standardised and structured approach to data storage and retrieval by defining relationships between separate tables.
- **SQL:** SQL or Structured Query Language is the standard language used by RDBMS to manage a database.

The installation of the MySQL RDBMS and starting the MySQL Command Line Client have been covered in the previous lesson. For a detailed guide on the installation process, refer to the QR Code provided below:

Activity

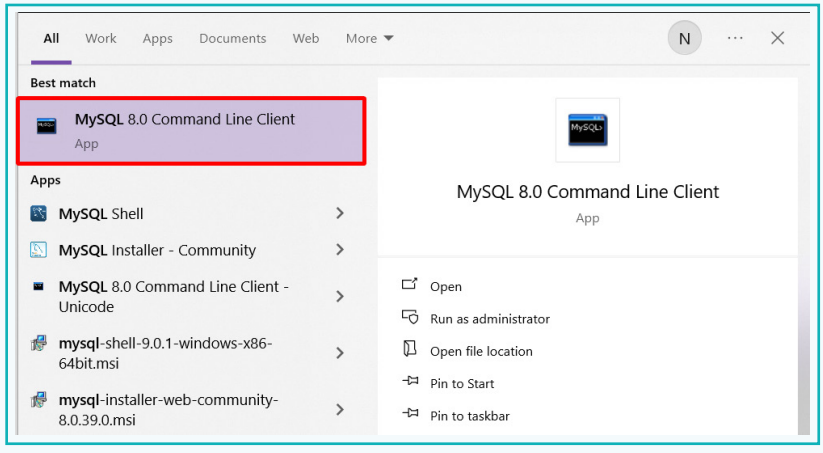
Create a simple **Student-Project** system that involves designing a database to manage Student Information, and Projects Undertaken, using Foreign Keys and Joins.

This database system will consist of 2 tables:

1. **Students Table:** Contains basic details of all students.
2. **Projects Table:** Stores details of projects, specifying which student is working on which project.

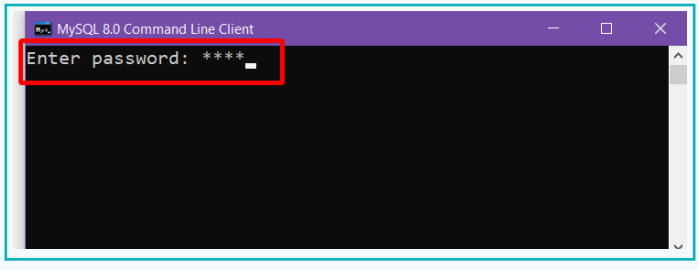
Part 1 - Create a Database

1. Open 'MySQL Client' from the 'Start' menu. This is the 'Command Line Interface' used for interacting with a database.



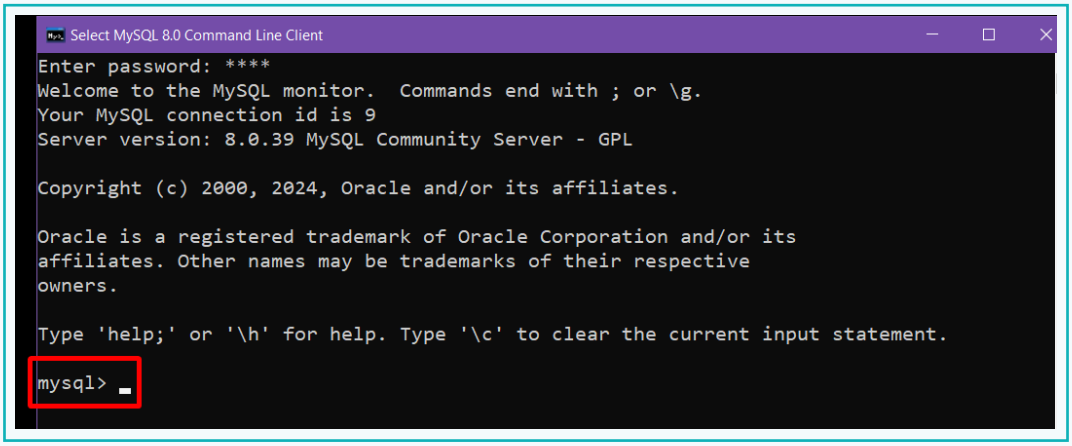
2. Enter the password for MySQL when prompted.

Note:
This password has been set during the MySQL installation process.

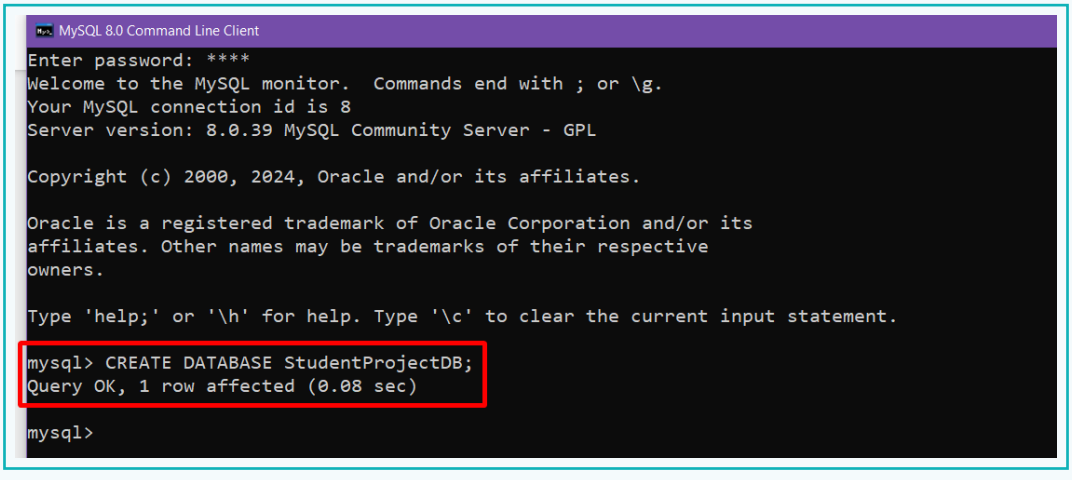


Activity

3. After entering the password, the MySQL Client window will appear. Commands can be entered here to perform various database operations.



4. To create a database, use the *CREATE DATABASE* command along with the database name. Ensure the command ends with a semicolon (;). Press the 'Enter' key after the command.



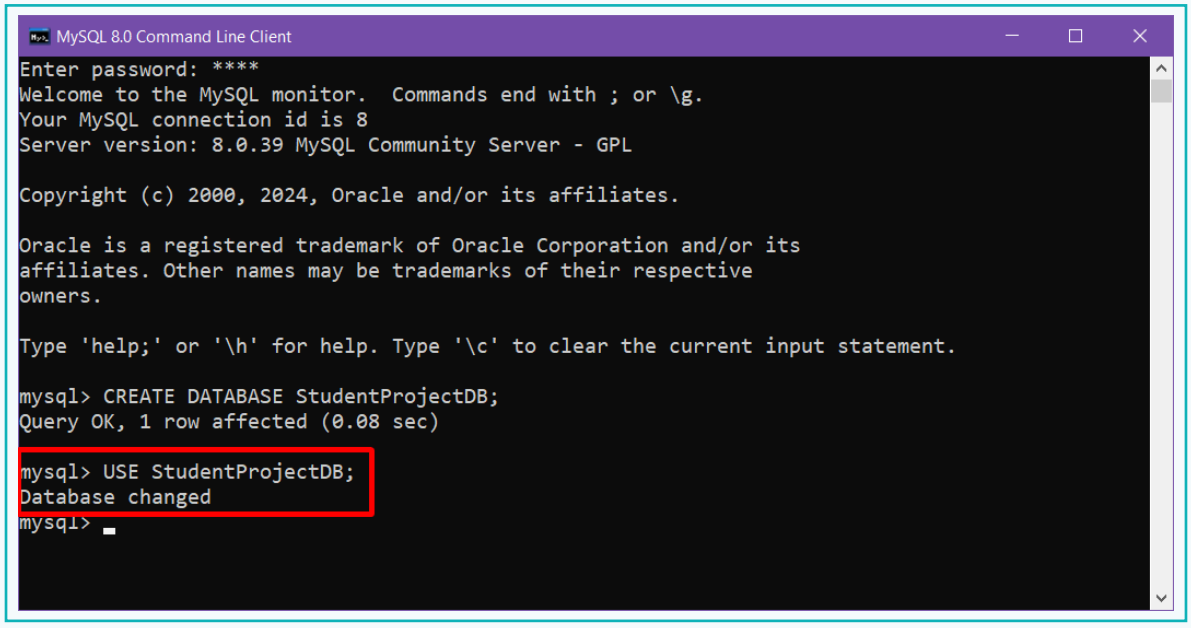
Note:
SQL Commands are case-insensitive, which means the commands can be written in both lower and upper case.

Activity

5 To specify the database to be used, enter the *USE* command.

The syntax is *USE Database_name*;

- Press 'Enter' after typing the command.



Part 2 - Create the Students Table

To store data in a database, first, a table must be created. A **Table** is a collection of related data organised or stored in the form of rows and columns.

When creating a table, it is essential to specify not only the field names but also the type of data that will be stored in each field.

Common data types include:

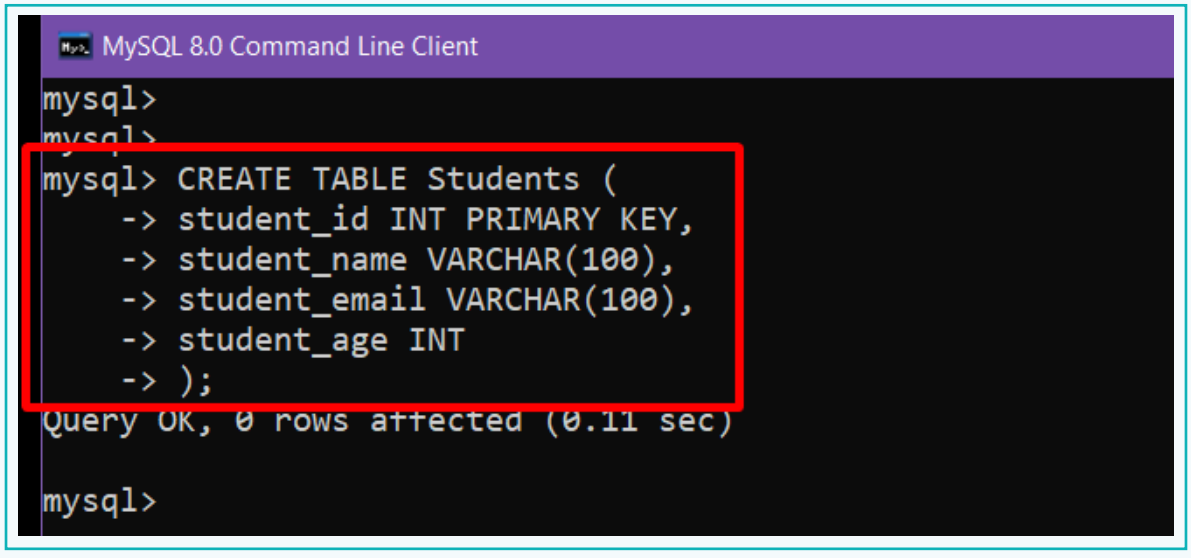
- **INT, FLOAT:** for storing numeric values.
- **VARCHAR, CHAR, TEXT:** for storing strings.
- **DATE, TIME:** for storing date and time.
- **BOOLEAN:** for storing Boolean values.

Activity

First, create a Students table to store the basic details of all students. Initially, include following four fields, but additional fields can be added as required.

- **Student ID**
- **Student Name**
- **Student Email**
- **Student Age**

1. Use the *CREATE TABLE* command in the MySQL Client to create the 'Students' table. Assign appropriate data types for each field and declare the *student_id* field as the 'Primary Key'.



Note:

- After declaring each field of the table, press the 'Enter' key to move to the next line.
- A **Primary Key** is a field (or combination of fields) in a database table that has unique, non-repeating values for each record. It uniquely identifies each record in the table and ensures that no two rows can have the same key value. It also cannot contain NULL values.

Activity

Part 3 - Insert and Store Data on the Students Table

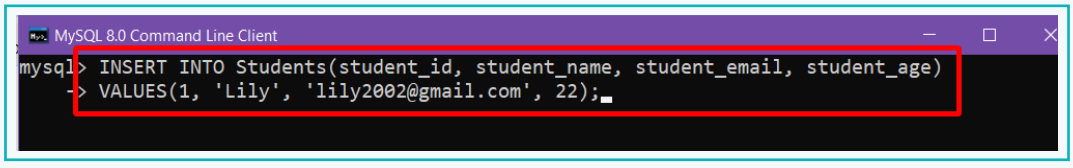
Now that the table has been created, the next step is to insert data into the table using the *INSERT* command.

Command syntax:

INSERT INTO table_name (field_1, field_2, ….. field_n)

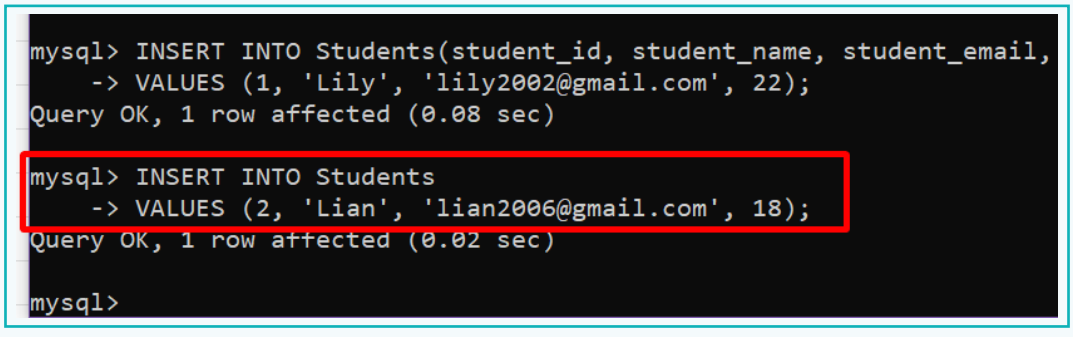
VALUES (value_1, value_2…value_n);

1. In the MySQL Client, execute the *INSERT* command to create the first record in the 'Students' table. For the *student_id* field, provide numerical values starting from 1. Press 'Enter' to move to the next line and provide the values.



Note:
It is not necessary to provide the field names along with the *INSERT* command if the values are specified in the correct order.

2. Insert another record without specifying the field names. Press 'Enter' once completed.



Activity

3 Any number of records can be stored on the table. For instance, create three more 'Student' records.

```
mysql>
mysql>
mysql> INSERT INTO Students VALUES (3, 'Meera', 'iammeera@gmail.com', 20);
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Students VALUES (4, 'Alex', 'alex8929@gmail.com', 21);
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Students VALUES (5, 'James', 'james@gmail.com', 22);
Query OK, 1 row affected (0.02 sec)

mysql>
```

Note:
The *INSERT* command can also be used in a single line instead of specifying the values on the next line.

4 Verify that the data is present in the table using the *SELECT* command. The asterisk (*) symbol is a wildcard operator that represents all fields in a table.

```
mysql> SELECT * FROM Students;
+------------+--------------+--------------------+-------------+
| student_id | student_name | student_email      | student_age |
+------------+--------------+--------------------+-------------+
|          1 | Lily         | lily2002@gmail.com |          22 |
|          2 | Lian         | lian2006@gmail.com |          18 |
|          3 | Meera        | iammeera@gmail.com |          20 |
|          4 | Alex         | alex8929@gmail.com |          21 |
|          5 | James        | james@gmail.com    |          22 |
+------------+--------------+--------------------+-------------+
5 rows in set (0.00 sec)

mysql>
```

Activity

Part 4 - Create the Projects Table with Foreign Key

To store the projects assigned to students, a separate table called **Projects** will be created.

The table, along with a sample record, will look like this:

Project_id	Project_name	Project_description
101	Web Development	Creating a simple website using HTML and CSS.

Projects table

This table will store details of the assigned projects and specify which student is working on a particular project. However, the 'Projects' table currently does not contain any student-related data. To link the 'Students' and 'Projects' tables, a relationship needs to be established. This is achieved using a **Foreign Key**, which ensures that each project entry is associated with a valid student from the 'Students' table.

A 'Foreign Key' is a field in one table that refers to the 'Primary Key' in another table. The table containing the foreign key is known as the **Child Table**, while the table containing the referenced primary key is called the **Parent Table**. In RDBMS, a foreign key is used to establish a relation and enforce a link between two tables.

Student_id	Student_name	Student_email	Student_age
1	Lily	Lily2002@gmail.com	22
2	Lian	Lian2006@gmail.com	18
3	Meera	Iammeera@gmail.com	20
4	Alex	Alex8929@gmail.com	21
5	James	James@gmail.com	22

Students table

Activity

Considering the above 'Students' and 'Projects' tables, the Primary Key field 'student_id' from the 'Students' table will be used as a foreign key in the 'Projects' table. This establishes a relation between the two tables, allowing for the identification of which project is assigned to which student, based on their 'student_id'.

So, modify the 'Projects' table to include the 'student_id' field, which will serve as the foreign key and enable tracking of project assignments.

The 'Projects' table should look something like this –

Project_id	Student_id	Project_name	Project_description
101	1	Web Development	Creating a simple website using HTML and CSS.

Now, by observing the 'Projects' table, it is clear that the student with 'student_id – 1', is working on the 'Web Development' project.

A Foreign Key is defined within the *CREATE TABLE* or *ALTER TABLE* command.

```
CREATE TABLE table_name (
column1 datatype,
column2 datatype,
....
FOREIGN KEY (column_name) REFERENCES referenced_table(referenced_column)
);
```

Note:
The datatype of the foreign key should be the same as the referenced primary key.

Activity

Consider the following table for better understanding. Notice how the 'student_id' field in the 'Projects' table acts as the Foreign Key, referencing the 'student_id' field in the 'Students' table.

Students Table

Student_id	Student_name	Student_email	Student_age
1	Lily	Lily2002@gmail.com	22
2	Lian	Lian2006@gmail.com	18
3	Meera	Iammeera@gmail.com	20
4	Alex	Alex8929@gmail.com	21
5	James	James@gmail.com	22

Projects Table

Project_id	Student_id	Project_name	Project_description
101	1	Web Development	Creating a simple website using HTML and CSS

1. Create the 'Projects' table along with the foreign key declaration.

```
mysql> CREATE TABLE Projects (
    -> project_id INT PRIMARY KEY,
    -> studentID INT,
    -> project_name VARCHAR(100),
    -> project_desc TEXT,
    -> FOREIGN KEY (studentID) REFERENCES Students(student_id)
    -> );
```

Activity

In the 'Projects' table,

- *project_id* is declared as the primary key.
- *studentID* field is defined, which we will declare as the foreign key.
- The *studentID* foreign key references the primary key *(student_id)* in the 'Students' table. While naming the foreign key we can use the same name *(student_id)* as the referenced primary key or a different one like, *StudentID*. Both naming conventions are acceptable.
- The *project_name* and *project_desc* will contain details regarding the name of the project and a brief description of it.

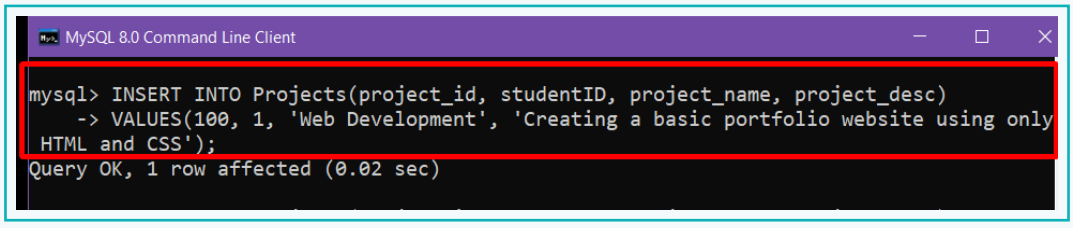
Notice the syntax for defining a Foreign Key: First, the keyword ***FOREIGN KEY*** is used, followed by the name of the foreign key field. Then, the keyword ***REFERENCES*** is specified, after which the 'Parent Table' name is provided along with its primary key.

Note:
Both VARCHAR and TEXT are used for storing strings or characters, but TEXT datatype is more suitable for longer text data, such as descriptions, articles, posts, etc, as it can store a greater number of characters.

Part 5 - Insert Records in the Projects Table

Understand that the 'Projects' table is created to store records of the project details along with the 'ID' of the student who is working on it.

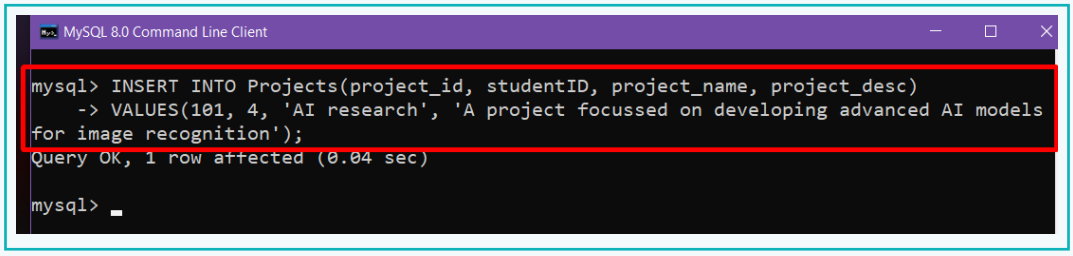
1. Assuming that 'Lily' is working on a project related to 'Web Development', insert a record into the 'Projects' table. For the *project_id* field, numerical values starting from 100 will be considered. Press 'Enter' at the end of the command.



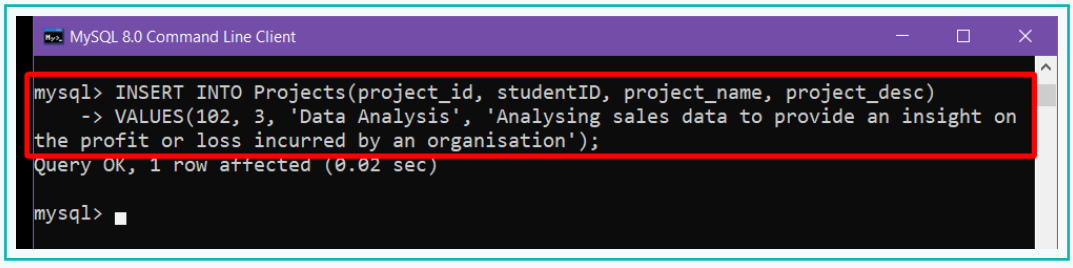
In this *INSERT* statement, the value '1' is provided for the *studentID* field, since the *student_id* of Lily is 1, as per the 'Students' table. This will associate Lily's record in the 'Students' table with this record, having *project_id - 100*.

Activity

2. Next, insert the project record for 'Alex', whose '*student_id*' is 4, assuming he is working on an 'AI-based' project.

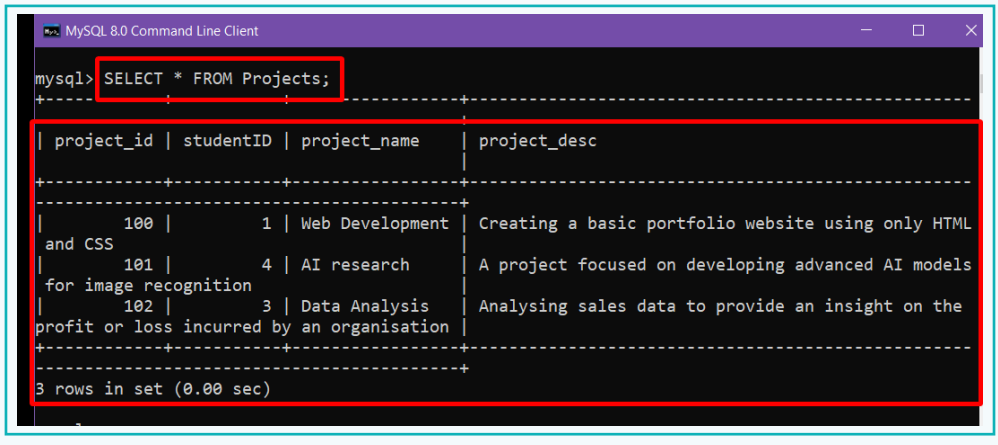


3. Create another record for 'Meera' (*student_id* – 3), who is working on a 'Data Analysis' project.



In this way, every time a student is assigned a project, the details can be stored in the 'Projects' table using the corresponding 'ID' of the student.

4. View all the stored records in the 'Projects' table using the *SELECT* statement.



Now that all records are stored, it is important to understand how to retrieve data from both the 'Students' and 'Projects' tables.

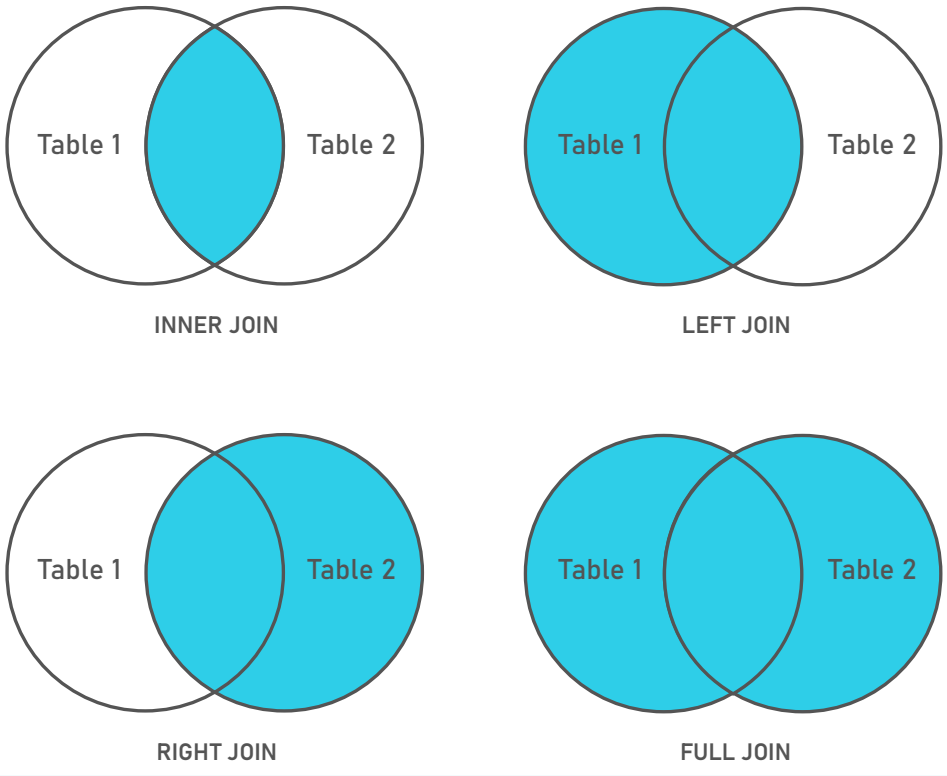
Activity

Part 6 - Retrieve data using Joins

Joins are one of the fundamental concepts in Relational Databases. They are used to retrieve data from two or more tables by combining records based on related column(s), typically with the help of primary and foreign keys. Joins enable the merging of rows from different tables based on a linked field, allowing for efficient data retrieval across tables.

Joins are of several types, as listed below:

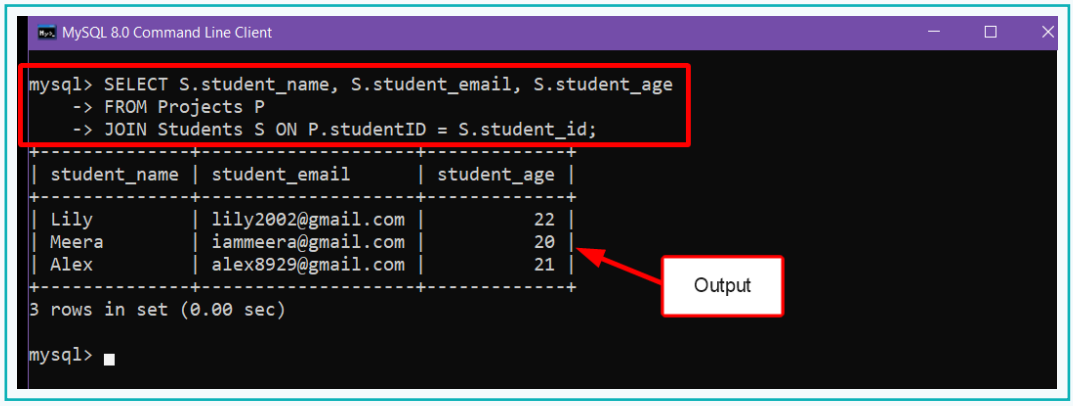
1. **Inner Join or Join:** Returns records that have a matching value in both tables.
2. **Left (Outer) Join:** Returns all records from the left table, and the matched records from the right table.
3. **Right (Outer) Join:** Returns all records from the right table, and the matched records from the left table.
4. **Full (Outer) Join:** Returns all records when there is a match in either the left or right table.



Let us implement Joins to retrieve data from multiple tables.

Activity

1. Retrieve records of only those students who are working on any project, by using Joins.



Explanation:

A. SELECT S.student_name, S.student_email, S.student_age

This clause specifies the columns or fields to retrieve. **'S'** here acts as an alias to the Students table for easier reference.

a. ***S.student_name***: The 'name' of the student from the Students table.
b. ***S.student_email***: The 'email' of the student from the Students table.
c. ***S.student_age***: The 'age' of the student from the Students table.

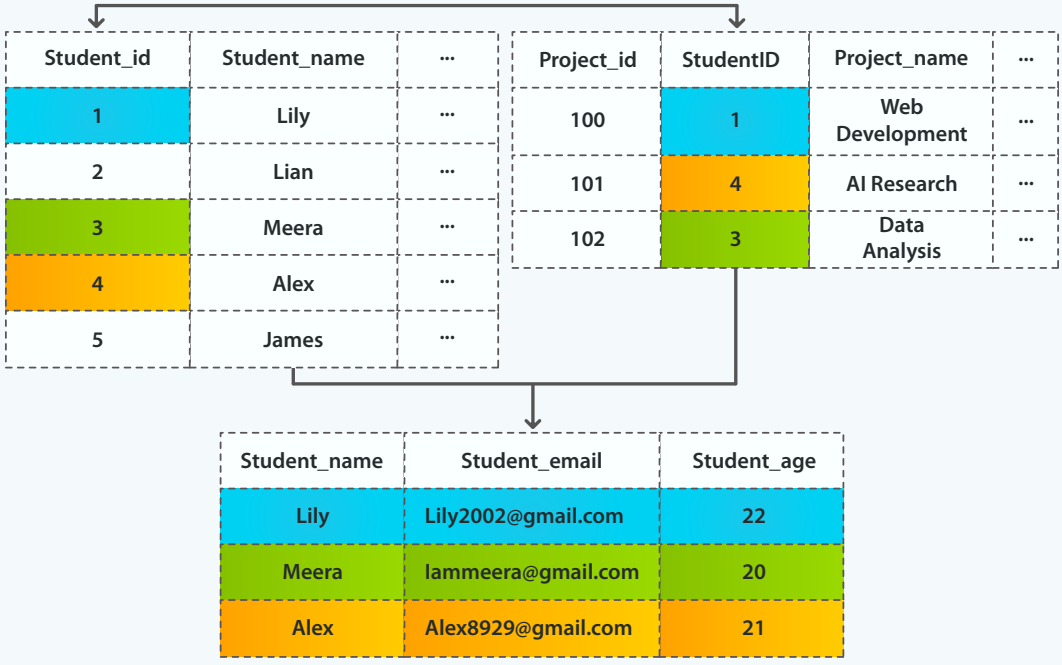
B. FROM Projects P

This indicates that the query will start with the Projects table. The **'P'** here acts as an alias to the Projects table for easier reference.

C. JOIN Students S ON P.studentID = S.student_id

This is an 'Inner Join' between the Projects and Students tables, using the *'student_id'* column as the common or linking column. **'S'** here acts as an alias to the 'Students' table.

Activity



Student_id	Student_name	...
1	Lily	...
2	Lian	...
3	Meera	...
4	Alex	...
5	James	...

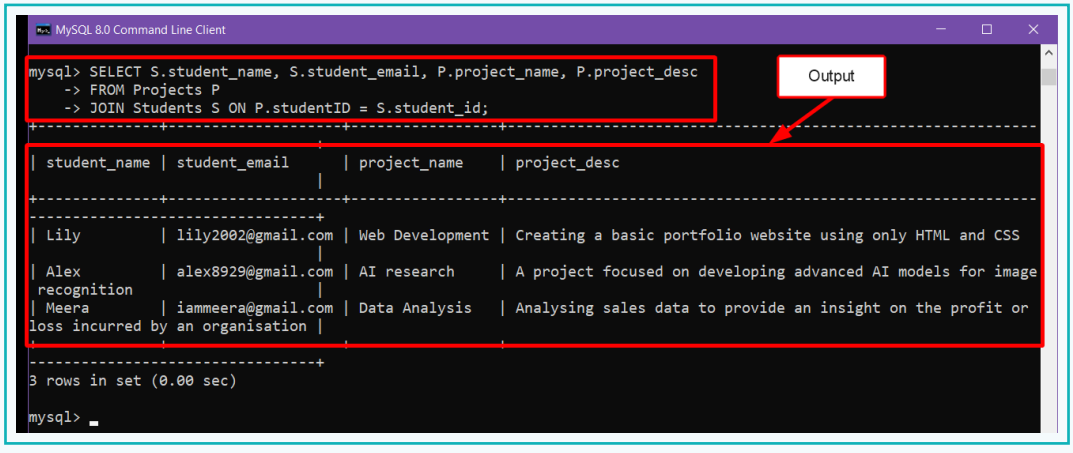
Project_id	StudentID	Project_name	...
100	1	Web Development	...
101	4	AI Research	...
102	3	Data Analysis	...

Student_name	Student_email	Student_age
Lily	Lily2002@gmail.com	22
Meera	Iammeera@gmail.com	20
Alex	Alex8929@gmail.com	21

2. Update the above query to include the project details, like the 'project name' and 'project description', along with the student details.

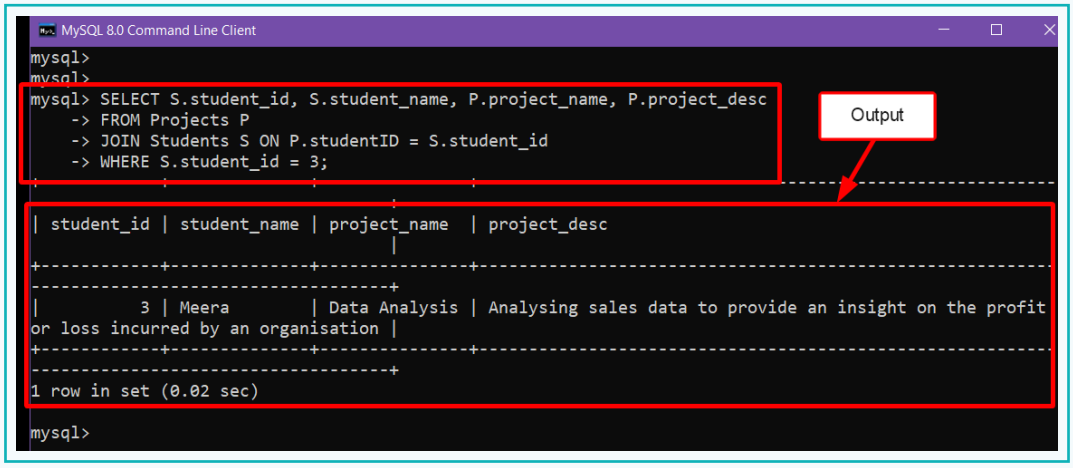
Note:

project_name and *project_desc* need to be prefixed with **'P'**, as these fields belong to the Projects table, having the alias **P**.

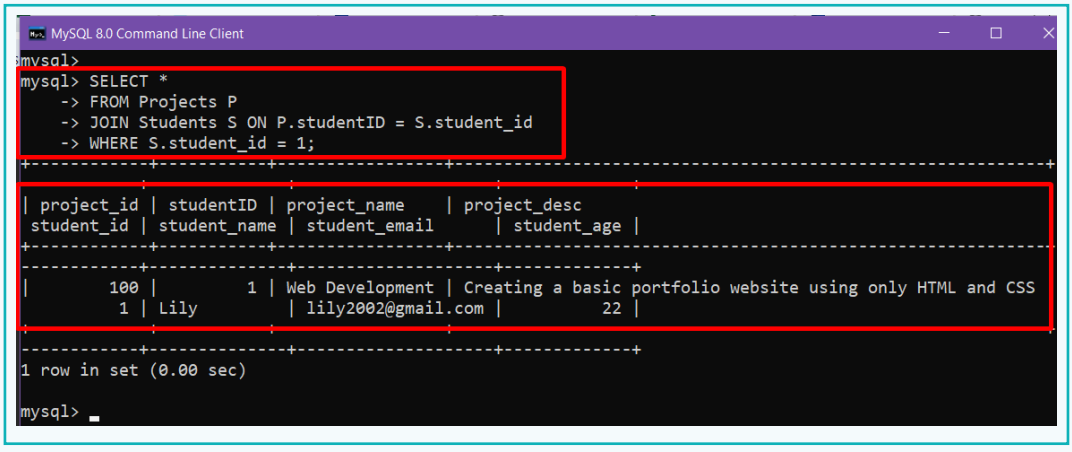


Activity

3. Use the *WHERE* clause to obtain records that include both student and project details for a specific student.



4. Use the wildcard operator *(*)* after the *SELECT* command to include all fields from both the 'Students' and 'Projects' tables.



In this way, using Joins, different types of data requests can be queried from multiple tables.

In this lesson, we revised the basic concepts of DBMS and learned intermediate concepts such as Joins and Foreign Keys. we also created a **'Student-Project'** system to further understand the implementation and usage of those concepts.

Exercise

Fill in the Blanks.

1. A _____ is a field in one table that references a Primary Key in another table.
2. To retrieve data from more than one table, _____ are used.
3. _____ returns records that have a matching value in both tables.

Tick mark the correct answer.

4. Which of the following keyword is used when creating a Foreign Key?

   a. REFERENCES b. SELECT

   c. PRIMARY KEY d. USE

State if the statements are True or False.

5. Numeric values are placed within single (‘ ’) or double quotes (“ ”).

   _____

6. Joins can be used to get data from more than one table.

   _____

05. Joins and Foreign Keys

Tech Flash

DBMS or Database Management System	:	A software package that enables users to create, maintain, and use a database.
Table	:	It is a collection of related data organised in the form of rows and columns.
Foreign Key	:	It is a field in one table that references the Primary Key in another table.
Joins	:	It is used to retrieve data from two or more tables by combining records based on related columns, typically with the help of primary and foreign keys.